Engineering Notebook — Project Start

Date: Oct. 23, 2026, 8:20 PM

Phase: Pre-design

Revision: 0.1

History

In 1998, two mathematicians —Marlow Anderson and Todd Feil— discovered a method of solving Lights Out puzzles using linear algebra. Their work can be found here: (https://www.jstor.org/stable/2690705).

Main goal

My goal with this project is to turn Anderson and Feil's mathematical algorithm into a C++ program, then design a physical machine designed to solve Lights Out puzzles using said program.

Objective

Make a "project requirements" Word document that outlines the Lights Out solver project.

Approach

Outline the project by brainstorming ideas in these categories:

- Objective
- Functional requirements
- Design constraints
- Verification criteria
- Open design decisions

Next steps

- Start working on the C++ program
- Create a function in C++ that turns the Lights Out matrix into a toggle matrix

Sign off: Oct. 23, 2026, 10:35 PM -Maiko Llanes

Engineering Notebook — Toggle Matrix

Date: Oct. 24, 2026, 5:55 PM

Phase: Pre-design

Revision: 0.2

Objective

Design a function in C++ that creates a toggle matrix using the original lights out matrix as the input.

Definition

The first step of solving a Lights Out puzzle is to find the toggle matrix. It is pertinent to first define what a toggle matrix is. Consider a 3 x 3 Lights Out puzzle; place the puzzle into a matrix, giving each tile a unique number from 1 to 9 in this configuration:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

To create the first row of the toggle matrix, we first check if toggling element 1 toggles element 1. Since this is true, we add an element with the value of 1 to our new matrix. Next, since toggling element 1 also toggles element 2 we add another 1 to the end of our matrix. In general, if toggling element 1 toggles element j, where j is the current element we are comparing 1 to, we add 1 to the end of our new row. If element 1 does not affect element j, we add a 0 to the end of the row instead. Following this process to the end of our original matrix will result in the row: (1 1 0 1 0 0 0 0 0). To create the following rows, we follow the same process with the next element, 2, then 3, until we use the process on element 9, resulting in the matrix:

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

This 9 x 9 matrix is our toggle matrix for our 3 x 3 Lights Out puzzle. And in general, for any $n \times n$ Lights Out puzzle, there exists a toggle matrix that is $n^2 \times n^2$ made up of only 0 and 1.

Approach

To determine where to place 0 and 1 in the toggle matrix, we need to find a way to identify if the position of element i is the same, or adjacent to, the position of the iterating element, j; for simplicity, I will refer to this condition as j being a neighbor of i. Assuming this time that the Lights Out matrix starts at position 0 instead of 1, j is:

- A neighbor of i if $j = i$.
- An "upstairs" neighbor of i if $i - rowsize \geq 0 \ \& \ j = i - rowsize$.
- A "downstairs" neighbor of i if $i + rowsize < matrixsize \ \& \ j = i + rowsize$.
- A "left" neighbor of i if $(i-1)\mathrm{mod}(rowsize) = i\mathrm{mod}(rowsize) - 1 \ \& \ i - 1 > 0 \ \& \ j = i - 1$.
- A "right" neighbor of i if $(i+1)\mathrm{mod}(rowsize) = i\mathrm{mod}(rowsize) + 1 \ \& \ j = i + 1$.

With this knowledge it is now possible to make a function in C++ that creates a toggle matrix.

Code changes

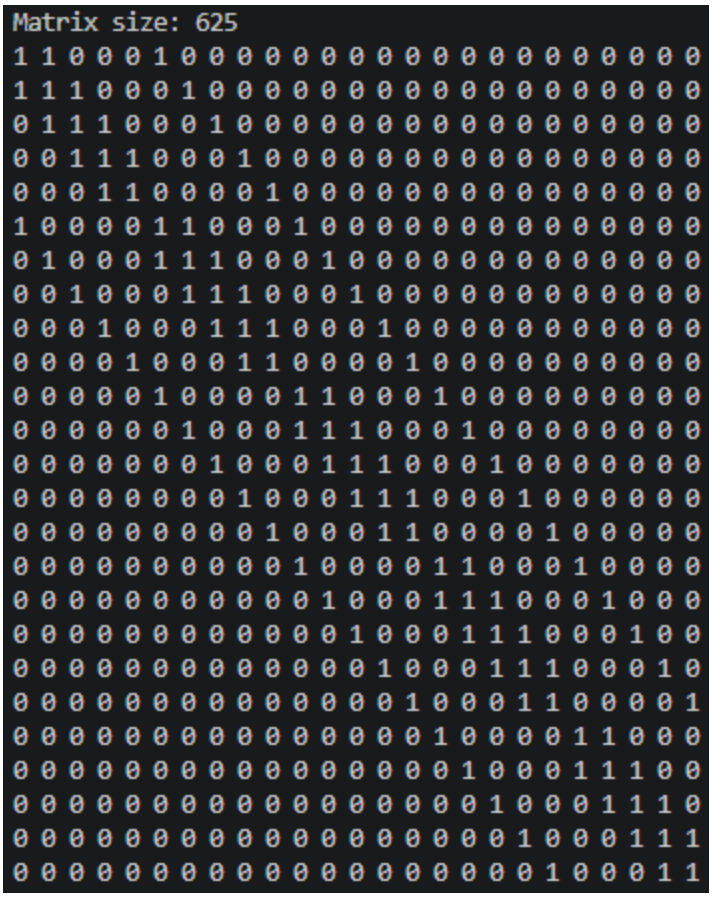
The toggle matrix function is finished; the full code can be found on my GitHub repository.

Verification

If inputting a 3×3 matrix results in its toggle matrix, the program is confirmed to work as intended thus far.

```
Matrix size: 81
1 1 0 1 0 0 0 0 0
1 1 1 0 1 0 0 0 0
0 1 1 0 0 1 0 0 0
1 0 0 1 1 0 1 0 0
0 1 0 1 1 1 0 1 0
0 0 1 0 1 1 0 0 1
0 0 0 1 0 0 1 1 0
0 0 0 0 1 0 1 1 1
0 0 0 0 0 1 0 1 1
```

The result lines up with the toggle matrix we calculated earlier; therefore, the program is working as intended. Since my machine will have a 5×5 grid, I also tested what a matrix of that size will look like by using this program:



Next steps

- Create functions in the program that perform row operations on the toggle matrices using mod(2) arithmetic.

Sign off: Oct. 24, 2026, 9:16 PM Maiko Llanes

Engineering Notebook — Row Operations

Date: Oct. 29, 2026, 1:04 AM

Phase: Pre-design

Revision: 0.3

Objective

Create row operation functions in the source code to manipulate the toggle matrix.

Approach

If you manipulate the toggle matrix into reduced row echelon form (RREF), the final column will represent the solution to the original lights out puzzle. To achieve RREF, you need two row operations: row addition & row swapping. An extremely important thing to note on row addition in our toggle matrix is that operation behaves exactly like XOR, so adding 1+1 results in 0 instead of 2.

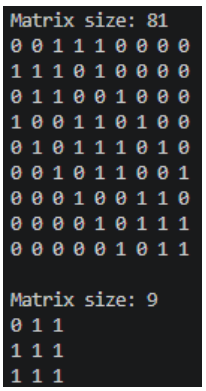
Code changes

Created xorRow function.

Verification

Starting with an 3×3 lights out puzzle with every light on:

- XORing row 2 to row 1 of the toggle matrix should result in row 1 changing to (0 0 1 1 1 0 0 0 0).
- The Lights Out matrix will have a 0 only in its 0th position.



The result matches expectations.

Code changes

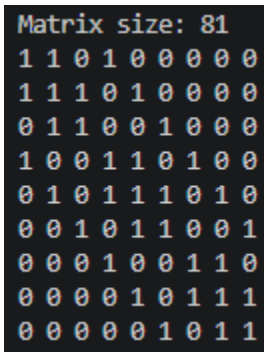
Created swapRow function.

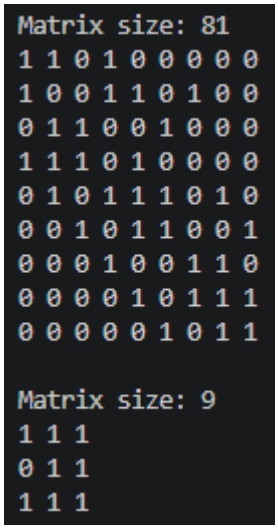
Verification

Starting with an 3×3 Lights Out puzzle with every light on besides position 1 (starting from position 0):

- Swap rows 2 and 4 of the toggle matrix.
- Visually compare the result to the original toggle matrix.

Original toggle matrix is the first image:





The result matches expectations.

Next steps

- Manipulate the toggle matrix into RREF.
- Determine if the Lights Out puzzle has a solution.
- If a solution exists, solve the puzzle.

Sign off: Oct. 29, 2026, 3:05 AM Maiko Llanes